

如何让 AI 更“听话” | Rules 高效使用指南

原创 TRAE TRAE.ai 2026年1月30日 17:00 北京



本文作者：简瑞，TRAE 技术专家

概述

在 AI Coding 场景中，Rule 的价值很直白：**让模型遵循开发者意愿**。你希望它输出什么风格、遵守哪些工程约束、在什么场景下做什么事——Rule 就是把这些“隐性习惯”变成“可显式执行”的表达。

在 TRAE 过去的版本里，我们支持 [配置 Project Rules 和 User Rules](#)。但随着项目复杂度提升、任务难度增加，大家很快会遇到一个现实问题：

单条 Rule 根本不够用！

规则写短了不够用；

写长了又变成一团乱麻；

更麻烦的是——你很难控制它在“什么时候该生效、什么时候不该生效”。

这个问题落到一个更本质的层面：我们需要的不再是一条“写得很全的规则”，而是一套**可维护、可复用、可控生效**的规则体系。

TRAE 的 Rules 能力升级后引入了**多规则管理**（便于拆分与维护）与**精细化生效**（便于控制使用范围与时机），同时支持导入 **AGENTS.md / CLAUDE.md**（降低迁移成本）。

本文会从最佳实践出发，系统讲清楚：

- 为什么一条 Rule 写太长会“反噬”效果？
- 多 Rules 应该怎么拆，才能可维护、可复用？
- 4 种生效方式怎么选，才能既听话又不打架？
- User Rule 为什么优化为改成“输入框并列编辑”，以及怎么用更舒服？
- 如何复用 AGENTS.md / CLAUDE.md，降低迁移成本？



单条 Rule 的三大问题：规模、冲突、时机

很多人第一次写 Rule 都会经历同一条路：“我要把经验都写进去，越全越好。”

但当 Rule 变得越来越长，问题会越来越明显，通常集中在三个方面：

1. 规模问题：越写越长，维护成本指数级上升

一条 Rule 里揉进代码风格、目录约定、框架规范、业务流程、排查 SOP ... 你会很快发现：

- 想改一点点内容，要在文件里翻半天
- 团队协作时很难分工维护，容易互相覆盖
- 规则更新很频繁时，内容会迅速腐化

2. 冲突问题：不同场景的规则揉在一起，AI 很容易“抓错重点”

比如你把“React 组件规范”和“Git Commit Message 模板”写在同一条 Rule 里。当你只是在问一个 UI 问题时，模型可能突然开始输出 commit message 结构；当你要写提交信息时，模型又开始强调 hooks 的最佳实践。

不是模型不聪明，而是你给它的规则没有边界。

3. 时机问题：你很难决定“这段规则什么时候该生效”

一些规则应该始终生效（比如输出语言、格式约定）；另一些只在特定文件或特定任务里才有意义（SQL 迁移、日志排查）。如果你没法控制规则生效时机，就会发生：

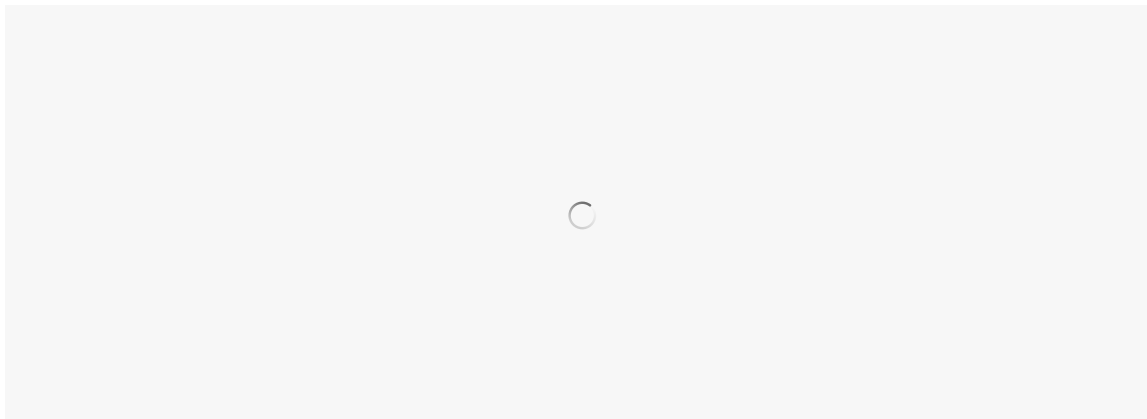
- 规则到处乱入，造成噪音与冲突
- 为了避免干扰，你又不敢写“关键但偶尔使用”的规则
- 最终 Rule 退化成“鸡肋”：写了很多，但帮不上什么忙



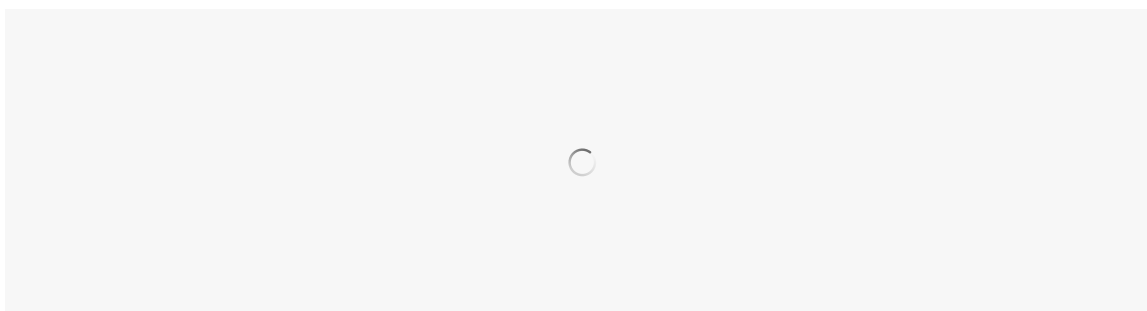
如何解决“规模化后的可控性”

为了解决上面的问题，TRAE 针对 Rules 能力进行了升级，把“规则”从一段长文本，升级成一套可拆分、可管理、可控生效的体系。你会看到两条主线能力：

1. **多规则管理**：允许配置多条 Project Rule 文件，把不同主题/职责拆到不同文件里



2. **精细化生效**：针对每条规则提供更细粒度的生效方式，用来控制适用范围与时机



多规则管理：把规则拆成模块，而不是把一条规则写成百科全书

当规则开始规模化，核心矛盾不是“写不下”，而是“管不住”。多规则的价值体现在三个方面：

- **可维护**：每条规则只管一类事情，修改更集中，不需要在文件里查找和定位
- **可协作**：团队可以按职责分工维护不同规则文件，减少互相覆盖
- **可组合**：不同项目/子项目可以复用同一组模块（如前端规范模块、后端规范模块、工作流模块）

实践上，我们建议把规则文件当作“模块”来设计：一条规则只解决一个主题/职责，并尽量保持短、具体、可执行。

精细化生效：规则不只是“写了就带上”，关键是“该出现时出现”

多规则解决了维护性问题，但要解决“冲突”和“噪音”，还需要控制规则在什么范围/什么时候生效。

为此，Rules 2.0 为 Project Rule 提供 4 种生效方式：

- 始终生效 (Always Apply)
- 指定文件生效 (Apply to Specific Files)
- 智能生效 (Apply Intelligently)
- 手动触发生效 (Apply Manually)

你可以把它们理解成不同的“控制粒度”：

- 控制范围：只对哪些文件/目录生效
- 控制时机：只有相关任务才生效，或者必须点名才生效

4 种生效方式怎么选：一张表快速决策

生效方式	适合放什么规则	典型例子
始终生效	低冲突、强一致、任何任务都不应违背的规则	输出语言/格式要求、通用代码风格、通用命名习惯、团队协作约定
指定文件生效	边界清晰、与文件类型/目录强相关的规则	**/*.sql 迁移规范、apps/web/** 前端规范、apps/api/** 后端规范
智能生效	偶尔使用但重要，希望“相关时自动出现”的规则	问题排查 checklist、性能优化流程、特定领域的注意事项
手动触发	高风险/高成本/误触发会严重干扰的规则	发布/回滚 SOP、敏感操作流程

决策原则：

- 越“通用且低冲突”，越偏向始终生效；
- 越“边界清晰”，越偏向指定文件生效；
- 越“偶发但关键”，越偏向智能生效；
- 越“高风险高干扰”，越应该手动触发。

小技巧：在输入框中新增 #Rule 能力，任何 Rule 文件都可以手动 mention，以确保本次会话模型一定参考该规范。



规则拆分方法论

下面是一套 Rule 拆分方法论，让每条 Rule 都短、准、可维护：先分层，再单一职责，最后注意要长期维护治理。

先按“职责分层”拆：从稳定到变化，从通用到业务

建议把 Project Rules 拆成 4 层：越稳定越通用的规则越靠前，越易变越业务的规则越靠后。

L0：协作/输出规范（稳定、通用）

- 解决“所有任务都要一致”的部分：输出语言、格式、代码块风格、注释习惯、基本协作约定等
- 推荐生效方式：始终生效

L1：技术栈/工程规范（中等稳定）

- 解决“某个子项目/技术栈内要一致”的部分：React/Vue 约定、Go 工程结构、目录分层、错误处理方式等
- 推荐生效方式：指定文件生效（按目录或文件类型）

L2：业务/领域规则（变化快）

- 解决“某个业务域里不能踩坑”的部分：订单/支付/风控的边界条件、关键字段、流程约束等

- 推荐生效方式：**指定文件生效或智能生效**（取决于是否能明确范围）

L3：工作流/SOP（偶发但高价值）

- 解决“只在特定任务里出现，但出现就很关键”的部分：问题排查、性能优化、发布/回滚检查清单等
- 推荐生效方式：**智能生效或手动触发**

再做“单一职责”拆分：一条 Rule 只管一件事

这条 Rule 能不能用一句话说清楚它在约束什么行为？不能，就继续拆。

你会发现很多“过长 Rule”的根源不是内容多，而是它同时在管三类东西：

- **规范类**（必须遵守的约束）
- **偏好类**（建议/倾向）
- **流程类**（按步骤做事）

把它们揉在一起，AI 很容易抓错重点。拆开之后，不仅更短，也更稳定。

3 个常用拆分维度：职责 / 范围 / 风险

实际落地时，除了分层，可以用三个维度快速决定“要不要拆成单独一条 rule”。

A. 按职责拆（最常用）

例如把“React 组件规范”和“状态管理规范”分开；把“错误处理规范”和“日志规范”分开。

B. 按范围拆（为指定文件生效服务）

只要你能清晰写出“这条规则属于哪个目录/哪些文件类型”，就值得拆出来配成指定文件生效。本条在 monorepo 的场景下收益尤其大。

C. 按风险拆（为手动触发服务）

凡是“误触发代价很大”的内容，都建议拆出来做成手动触发：发布/回滚、敏感操作等。

规则资产治理：让多 Rules 长期不劣化

写好的 Rule 如果不注意维护治理，可能在后期迭代时越用越乱：不断往里加内容，但没人删；最后 Rule 越来越臃肿，影响整个团队的任务执行效果。

为了避免建设好的 Rule 体系劣化：

1. 什么时候加？

- 新增规则之前，先想清楚它属于哪一类、应该用哪种生效方式（参考前文的拆分方法）
- 能圈范围就不要写成全局，能作为流程就不要混进工程规范里，个人偏好更不要塞进项目规则。

2. 什么时候删？

当你发现某条 Rule 持续把任务带偏（比如只是问 UI 问题却经常被引导去输出一整套文档结构、或者频繁强调不相关的流程），就要优先怀疑这条规则已经变成“噪音源”。

- 如果它本质是个人习惯/表达偏好：从 Project Rule 里删掉，放回 User Rule
- 如果它只在少数场景成立：不要硬删，改成智能生效或手动触发（让它默认不出现）
- 如果团队已经不再认可或场景消失：直接删掉，避免“历史包袱”常驻上下文

3. 什么时候改？

- 如果规则仍然正确，但出场时机不对：改生效方式
- 如果规则方向对，但写法太虚/太长，模型执行不稳定：改结构（可执行约束 + 动作）

一个实用技巧：如果你在加规则时担心——“为了不被这条规则打扰，有些任务我都不敢用”——那它不应该放在 Always 里。把它降级到智能生效或手动触发，让它“该出现时出现”，才不会反噬整体体验。



多 Rules 实践：从 0 到 1 快速上手

前面我们介绍了 Rule 配置的方法论，这一节我们结合具体示例讲讲如何落地。

如果你的项目中还没有配置任何 Rule，推荐先配置 3 条最小版本的可用规则，覆盖常见诉求

1. 项目介绍与协作约定（始终生效）

这条规则只负责提供“项目级上下文信息”：项目做什么、关键目录在哪里、怎么本地运行、改动后的基本验证方式。它的目标是让模型**不需要每次都重新问一遍项目背景**，并且减少“改错地方/跑不起来/漏验证”的概率。

```
---
alwaysApply: true
---

# 项目概览

## 项目做什么
- {{一句话描述：产品/服务的核心功能}}

## 技术栈与关键约定
- 前端：{{React/Vue/Next.js...}}；后端：{{Go/Node/...}}；包管理：{{pnpm/yarn/...}}
- 代码规范/工具：{{eslint/prettier/gofmt/...}}

## 目录速览
- apps/web/**：{{前端应用入口}}
- apps/api/**：{{后端服务入口}}
- packages/**：{{共享库/组件库}}
- docs/**：{{文档/规范}}

## 本地开发与调试
- 安装：{{pnpm i / go mod tidy ...}}
- 启动：{{pnpm dev / make dev / docker compose up ...}}
- 常用脚本：{{lint/test/build}}

## 改动要求
- 修改前：确认影响范围（涉及哪些模块/接口/页面）
- 修改后：至少完成 {{基本验证项：lint/test/关键页面回归/接口联调}} 中的 {{N}} 项
- 输出代码时：优先给出“最小 diff + 验证步骤”，避免大段背景说明
```

2. 技术栈隔离（指定文件生效）

给前端项目指定开发约束，确保前端任务符合统一规范。

```
---
alwaysApply: false
globs: apps/web/**, **/*.tsx
---
```



```
# React 开发约束
```

- 组件拆分与命名：...
- hooks 约定：...
- 状态/请求/错误处理：...
- UI 边界处理：空态/loading/超长/错误态

3. 高风险流程（手动生效）

给常用的工作流配置要求，默认不生效，仅可手动指定

```
---  
alwaysApply: false  
---
```

```
# 发布/回滚清单
```

发布步骤：

```
- ...
```

验证清单：

```
- ...
```

回滚条件与步骤：

```
- ...
```

指定文件生效常用写法

指定文件生效模式是减少 Rule 冲突最直接的手段。

常用的 Globs 模式：

- Monorepo 子项目隔离：***apps/web/*****、***apps/api/*****、***packages/ui/*****
- 按文件类型生效：*****/*.tsx***、*****/*.sql***、*****/*.md***
- 按目录圈范围：***src/*****、***migrations/*****、***docs/*****

实际落地时，不一定要追求极致精准，可以先用目录圈住范围，再补充文件类型。如果生效范围还是太大，再考虑复杂的 Globs 匹配方式。

智能生效触发词

智能生效 Rule 的描述，建议写具体的“高信号词”，避免写大而空的“泛词”

- 要写“页面白屏 / 报错堆栈 / 接口超时 / 布局错乱”等具体场景，不要写“遇到问题时”

- 要写“性能变差 / 卡顿 / FPS”，不要写“需要优化时”

泛词看似覆盖范围大，实际可能导致规则在大量无关对话中出现，增加噪音。

实践效果对比

假如你已经正确合理的配置了 Rule，配置前后会有哪些变化？

1. Case 1：排查类问题——从“发散建议”到“按清单执行”

场景：页面布局错乱，按钮点不到了

没有 Rule 时

- AI 回答发散：从 CSS 到 React 到浏览器兼容性，什么都说
- 缺少排查顺序和信息采集
- 你要手动追问：先做什么再做什么

配置 Rule 之后

- AI 根据智能生效 Rule，按照排查清单执行，采集关键信息（环境、相关代码、控制台报错
- 给出排查路径：定位范围 -> 验证假设 -> 给修复方案
- 按照规则自行验证 + 回归

2. Case 2：发布/回滚——从“容易漏项”变成“默认不漏的清单”

场景：准备发版，让 AI 进行步骤流程检查

没有 Rule 时

- AI 给一套看似完整的流程，但和实际团队规范不符
- 容易遗漏关键点，每次回答内容不稳定

配置 Rule 之后

- 按照约定的发布流程清单，逐一检查

如果你不确定怎么开始，可以按照以下顺序：

- 先写一个始终生效的全局规则
- 按目录拆几个“指定文件生效”的技术栈规则
- 等团队用习惯了，再补“智能生效”的特定场景规则。

规则不是一次写完的，而是像工程配置一样，边用边迭代。



个人 Rules 管理

Project Rules 解决的是“团队/项目的一致性”。但每个人还有一些**个人偏好**：表达方式、解释粒度、默认行为习惯等，更适合放在 **User Rules**，避免把个人习惯塞进项目规则里造成干扰。

支持输入框编辑

过去编辑 User Rule 需要打开编辑器，更像在做工程配置；现在我们支持**多条 User Rule**，并在规则面板里以**一条条并列输入框**呈现，直接编辑、增删、调整都更顺手。



User Rule 更强调“随手可改、一眼可见、好维护”。

写什么最合适

建议只放三类“个人层”的内容：

1. 输出与沟通偏好

- 默认用中文/英文
- 解释方式（只给要点 / 先结论后步骤）
- 输出结构（是否给 checklist、是否给验证步骤）

2. 默认行为偏好

- 改代码前先简述计划与影响范围
- 改完后给验证方法/回归点

3. 个人工作习惯（可选）

- 你常用的排查方式、日志偏好等

User Rule 尽量写个人偏好，不要覆盖团队约定。



擅用 AGENTS.md 与 CLAUDE.md

很多团队已经在使用社区规范 *AGENTS.md*；也有不少人已经在使用 *CLAUDE.md*。核心价值很简单：

- **复用既有规则资产**：不需要因为换工具就重写一遍
- **降低迁移成本**：把历史沉淀快速带到 TRAE 的规则体系里继续用
- **与 Rules 互补**：通用规范可直接复用，项目内更精细的“范围/时机控制”仍建议用 TRAE 的多 rules + 生效方式来实现



总结：把规则写成“能维护的资产”

- 多规则不是为了写更多，而是为了把边界写清楚。
- 让规则该出现时出现：范围隔离冲突，时机控制噪音。
- 把规则当成工程资产：短、准、可维护，才能长期让 AI 更听话。

本文分享的多规则管理能力暂时仅在 TRAE 国际版支持，中国版敬请期待。

你还有哪些好用的 Rules 使用技巧？希望 TRAE 未来支持哪些 Rules 相关能力？欢迎在评论区留言和我们交流！



TRAIE.ai 

TRAIE, The Real AI Engineer | 字节跳动旗下的AI编程产品, 你的专属AI开发工程师... >
162篇原创内容

公众号

最佳实践 · 目录 ≡

< 上一篇

从“能用”到“会用” | 如何写好一个 Skill

下一篇 >

不止是写代码 | 如何用 TRAIE 进行复杂数据分析

